26.1 Cron 系统设计

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~200k tokens，输出 ~6k tokens（本节）

AI Agent 不应该只在用户主动对话时才工作。OpenClaw 的 Cron 系统让 Agent 可以**按时间计划自动执行任务**——每天早上发送新闻摘要、每小时检查邮箱、在指定时间提醒用户等。这一节分析 Cron 系统的核心设计：服务类、调度引擎、表达式解析和作业规范化。

26.1.1 Cron 服务（`src/cron/service.ts`）

类设计

`CronService` 是 Cron 系统的对外门面（Facade），它封装了所有定时任务的管理操作：

```typescript
// src/cron/service.ts

export class CronService {
  private readonly state;

  constructor(deps: CronServiceDeps) {
    this.state = createCronServiceState(deps);
  }

  async start()  { await ops.start(this.state); }
  stop()         { ops.stop(this.state); }

  async status() { return await ops.status(this.state); }
  async list(opts?: { includeDisabled?: boolean }) { ... }
  async add(input: CronJobCreate)  { return await ops.add(this.state, input); }
  async update(id: string, patch: CronJobPatch) { ... }
  async remove(id: string) { ... }
  async run(id: string, mode?: "due" | "force") { ... }

  wake(opts: { mode: "now" | "next-heartbeat"; text: string }) {
    return ops.wakeNow(this.state, opts);
  }
}
```

所有实际逻辑都委托给 `service/ops.ts` 中的纯函数， `CronService` 本身只是一个薄包装。这种设计便于测试——可以直接测试 `ops` 函数而不必实例化整个服务。

依赖注入

`CronServiceDeps` 定义了 Cron 服务所需的所有外部依赖：

```typescript
// src/cron/service/state.ts

export type CronServiceDeps = {
  nowMs?: () => number;                // 时间源（可注入假时钟）
  log: Logger;                         // 日志接口
  storePath: string;                   // 作业存储文件路径
  cronEnabled: boolean;                // 是否启用调度
  enqueueSystemEvent: (text: string, opts?) => void;  // 发送系统事件
  requestHeartbeatNow: (opts?) => void;               // 请求立即心跳
  runHeartbeatOnce?: (opts?) => Promise<HeartbeatRunResult>;  // 同步心跳
  runIsolatedAgentJob: (params) => Promise<{...}>;    // 运行隔离 Agent
  onEvent?: (evt: CronEvent) => void;                 // 事件回调
};
```

依赖	说明
`nowMs`	可注入的时间源，测试时可用假时钟精确控制时序
`enqueueSystemEvent`	向主会话的系统事件队列推送文本（如提醒消息）
`requestHeartbeatNow`	请求 Agent 立即唤醒并处理队列中的事件
`runIsolatedAgentJob`	在隔离会话中运行 Agent（用于独立的 Cron 任务）
`onEvent`	事件回调，Gateway 用它向客户端广播 Cron 事件

内部状态

```typescript
export type CronServiceState = {
  deps: CronServiceDepsInternal;
  store: CronStoreFile | null;      // 内存中的作业列表（从文件加载）
  timer: NodeJS.Timeout | null;     // 当前的 setTimeout 句柄
  running: boolean;                 // 是否正在执行作业（防止重入）
  op: Promise<unknown>;             // 操作序列化锁
  warnedDisabled: boolean;          // 是否已警告过禁用状态
  storeLoadedAtMs: number | null;   // 上次加载存储的时间
  storeFileMtimeMs: number | null;  // 上次加载时的文件修改时间
};
```

`store` 字段是关键——所有作业数据都以 JSON 文件存储在磁盘上（默认 `~/.openclaw/cron/jobs.json`），启动时加载到内存，修改后写回磁盘。这是一种"文件即数据库"的简单持久化方案，适合单用户场景。

启动流程

```
// src/cron/service/ops.ts — start

export async function start(state: CronServiceState) {
  await locked(state, async () => {
    if (!state.deps.cronEnabled) {
      state.deps.log.info({ enabled: false }, "cron: disabled");
      return;
    }

    // 1. 加载作业存储
    await ensureLoaded(state, { skipRecompute: true });

    // 2. 清除残留的"运行中"标记（进程上次可能异常退出）
    for (const job of state.store?.jobs ?? []) {
      if (typeof job.state.runningAtMs === "number") {
        state.deps.log.warn({ jobId: job.id }, "cron: clearing stale running marker");
        job.state.runningAtMs = undefined;
      }
    }

    // 3. 执行错过的作业（进程重启期间到期的任务）
    await runMissedJobs(state);

    // 4. 重新计算所有作业的下次运行时间
    recomputeNextRuns(state);

    // 5. 持久化状态
    await persist(state);

    // 6. 设置定时器
    armTimer(state);
  });
}
```

启动流程中有两个重要的恢复机制：

- **清除残留标记**——如果进程在作业执行期间崩溃，`runningAtMs` 标记会残留，需要在启动时清除
- **补跑错过的作业**——`runMissedJobs` 会找出所有 `nextRunAtMs` 已过期的启用作业并立即执行

操作序列化

所有操作通过 `locked()` 函数序列化，防止并发修改导致数据竞争：

```
// src/cron/service/locked.ts — locked（概念实现）

export async function locked<T>(state: CronServiceState, fn: () => Promise<T>): Promise
  // 将操作排队，确保前一个操作完成后才执行下一个
  const prev = state.op;
  const next = prev.then(() => fn());
  state.op = next.catch(() => {});
  return await next;
}
```

> **衍生解释——操作序列化**：在单线程的 Node.js 中，虽然没有真正的线程并发问题，但 async/await 可以导致交错执行。例如，两个并发的 `add()` 调用可能同时读取存储、同时修改、同时写回，导致一个操作的修改被覆盖。 `locked()` 通过 Promise 链确保操作按顺序执行，相当于一个异步互斥锁（Mutex）。

26.1.2 调度引擎（ `src/cron/schedule.ts` ）与 Croner 库

三种调度类型

OpenClaw 支持三种定时调度方式：

```
// src/cron/types.ts

export type CronSchedule =
  | { kind: "at"; at: string }                                  // 一次性：指定时间执行
  | { kind: "every"; everyMs: number; anchorMs?: number }       // 周期性：固定间隔
  | { kind: "cron"; expr: string; tz?: string };                // Cron 表达式
```

类型	示例	用途
`at`	`"2026-03-01T09:00:00Z"`	一次性提醒、定时任务
`every`	`everyMs: 3600000` （1小时）	固定间隔轮询

类型	示例	用途
`cron`	`"0 9 * * 1-5"` （工作日早9点）	标准 Cron 表达式

computeNextRunAtMs：下次运行时间计算

```
// src/cron/schedule.ts — computeNextRunAtMs

export function computeNextRunAtMs(
  schedule: CronSchedule, nowMs: number
): number | undefined {

  // 一次性调度
  if (schedule.kind === "at") {
    const atMs = parseAbsoluteTimeMs(schedule.at);
    return atMs !== null && atMs > nowMs ? atMs : undefined;
  }

  // 固定间隔调度
  if (schedule.kind === "every") {
    const everyMs = Math.max(1, Math.floor(schedule.everyMs));
    const anchor = Math.max(0, Math.floor(schedule.anchorMs ?? nowMs));
    if (nowMs < anchor) return anchor;
    const elapsed = nowMs - anchor;
    const steps = Math.max(1, Math.floor((elapsed + everyMs - 1) / everyMs));
    return anchor + steps * everyMs;
  }

  // Cron 表达式调度
  const cron = new Cron(expr, {
    timezone: resolveCronTimezone(schedule.tz),
    catch: false,
  });
  let cursor = nowMs;
  for (let attempt = 0; attempt < 3; attempt++) {
    const next = cron.nextRun(new Date(cursor));
    if (!next) return undefined;
    const nextMs = next.getTime();
    if (Number.isFinite(nextMs) && nextMs > nowMs) return nextMs;
    cursor += 1_000;  // 避免卡在边界上
  }
  return undefined;
}
```

every 类型的锚点机制：`anchorMs` 是间隔的参考起始点。下次运行时间通过 `anchor + N × everyMs` 计算（取大于 `nowMs` 的最小 N）。这确保了即使进程重启，间隔也是从原始

锚点计算，而非从重启时刻开始。

> **衍生解释——Croner**：Croner 是一个轻量级的 Cron 表达式解析库（纯 JavaScript，无依赖）。它支持标准的 5 字段 Cron 表达式（分 时 日 月 周），并提供 `nextRun()` 方法计算下次匹配时间。OpenClaw 使用它来解析 Cron 表达式，但不使用它的作业调度功能——调度逻辑由 OpenClaw 自己的 `armTimer` 实现。

时区处理

```
function resolveCronTimezone(tz?: string) {
  const trimmed = typeof tz === "string" ? tz.trim() : "";
  if (trimmed) return trimmed;  // 用户指定的时区
  return Intl.DateTimeFormat().resolvedOptions().timeZone;  // 系统默认
}
```

Cron 表达式的时区默认为系统时区，用户可通过 `tz` 字段指定 IANA 时区标识（如 `"America/New_York"` 、 `"Asia/Shanghai"` ）。

定时器机制

`armTimer` 设置下一次唤醒的 `setTimeout` ：

```
// src/cron/service/timer.ts — armTimer

const MAX_TIMER_DELAY_MS = 60_000;  // 最大延迟 1 分钟

export function armTimer(state: CronServiceState) {
  if (state.timer) clearTimeout(state.timer);
  state.timer = null;

  if (!state.deps.cronEnabled) return;

  const nextAt = nextWakeAtMs(state);  // 所有作业中最近的下次运行时间
  if (!nextAt) return;

  const delay = Math.max(nextAt - state.deps.nowMs(), 0);
  // 关键：限制最大延迟为 1 分钟
  const clampedDelay = Math.min(delay, MAX_TIMER_DELAY_MS);

  state.timer = setTimeout(async () => {
    await onTimer(state);
  }, clampedDelay);
}
```

为什么限制 1 分钟最大延迟？ 注释说明了原因："避免调度偏移，并在进程被暂停或系统时钟跳变时快速恢复"。JavaScript 的 `setTimeout` 在系统休眠（如笔记本合盖）后可能偏移很大，每分钟唤醒一次可以检测到这种情况并及时触发到期的作业。

26.1.3 Cron 表达式解析（`src/cron/parse.ts`）

`parse.ts` 负责解析各种时间格式为 Unix 毫秒时间戳：

```typescript
// src/cron/parse.ts — parseAbsoluteTimeMs

const ISO_TZ_RE = /(Z|[+-]\d{2}:?\d{2})$/i;
const ISO_DATE_RE = /^\d{4}-\d{2}-\d{2}$/;
const ISO_DATE_TIME_RE = /^\d{4}-\d{2}-\d{2}T/;

function normalizeUtcIso(raw: string) {
  if (ISO_TZ_RE.test(raw))        return raw;               // 已有时区：原样返回
  if (ISO_DATE_RE.test(raw))      return `${raw}T00:00:00Z`;  // 纯日期→当天 00:00 UTC
  if (ISO_DATE_TIME_RE.test(raw)) return `${raw}Z`;           // 无时区的日期时间→UTC
  return raw;
}

export function parseAbsoluteTimeMs(input: string): number | null {
  const raw = input.trim();
  if (!raw) return null;

  // 纯数字→直接当作 Unix 毫秒时间戳
  if (/^\d+$/.test(raw)) {
    const n = Number(raw);
    return Number.isFinite(n) && n > 0 ? Math.floor(n) : null;
  }

  // 尝试解析为 ISO 日期/日期时间
  const parsed = Date.parse(normalizeUtcIso(raw));
  return Number.isFinite(parsed) ? parsed : null;
}
```

支持的时间格式：

输入格式	示例	解析方式
Unix 毫秒	`"1709280000000"`	直接转换为数字
ISO 日期	`"2026-03-01"`	补全为 `2026-03-01T00:00:00Z`
ISO 日期时间（无时区）	`"2026-03-01T09:00:00"`	补全为 UTC（`2026-03-01T09:00:00Z`）
ISO 日期时间（有时区）	`"2026-03-01T09:00:00+08:00"`	原样解析

26.1.4 Cron 规范化（`src/cron/normalize.ts`）

问题：LLM 输入不可靠

Cron 作业通常由 Agent 通过工具调用创建。LLM 生成的参数可能：

- 大小写不一致（`"SystemEvent"` vs `"systemEvent"`）
- 使用旧版字段名（`atMs` vs `at`， `deliver` vs `delivery`）
- 缺少必要的默认值
- 字段位置不对（`model` 放在顶层而非 `payload` 内）

`normalize.ts` 的职责就是将这些"脏输入"规范化为统一的内部格式。

三层规范化

```
// src/cron/normalize.ts — 核心结构

export function normalizeCronJobInput(raw: unknown, options): UnknownRecord | null {
  const base = unwrapJob(raw);        // 1. 展开嵌套（data/job 包装）
  const next = { ...base };

  // 2. 规范化各个字段
  if (isRecord(base.schedule))  next.schedule = coerceSchedule(base.schedule);
  if (isRecord(base.payload))   next.payload = coercePayload(base.payload);
  if (isRecord(base.delivery))  next.delivery = coerceDelivery(base.delivery);

  // 3. 向下兼容：顶层字段迁移到 payload 内
  copyTopLevelAgentTurnFields(next, payload);
  copyTopLevelLegacyDeliveryFields(next, payload);
  stripLegacyTopLevelFields(next);

  // 4. 应用默认值（仅 create，不在 patch 时应用）
  if (options.applyDefaults) {
    if (!next.wakeMode) next.wakeMode = "now";
    if (typeof next.enabled !== "boolean") next.enabled = true;
    // ... 更多默认值
  }

  return next;
}
```

调度规范化

```
function coerceSchedule(schedule: UnknownRecord) {
  const next = { ...schedule };
  const rawKind = schedule.kind?.trim().toLowerCase();
  const kind = rawKind === "at" || rawKind === "every" || rawKind === "cron"
    ? rawKind : undefined;

  // 自动推断 kind
  if (!kind) {
    if (schedule.atMs || schedule.at)        next.kind = "at";
    else if (typeof schedule.everyMs === "number") next.kind = "every";
    else if (typeof schedule.expr === "string")    next.kind = "cron";
  }

  // atMs（旧版数字格式）→ at（新版 ISO 字符串格式）
  if ("atMs" in next) {
    next.at = new Date(parsedAtMs).toISOString();
    delete next.atMs;
  }

  return next;
}
```

负载规范化

```
function coercePayload(payload: UnknownRecord) {
  const next = { ...payload };

  // 修正大小写：agentturn → agentTurn, systemevent → systemEvent
  if (kindRaw === "agentturn")     next.kind = "agentTurn";
  if (kindRaw === "systemevent")   next.kind = "systemEvent";

  // 自动推断负载类型
  if (!next.kind) {
    if (next.message) next.kind = "agentTurn";
    else if (next.text) next.kind = "systemEvent";
  }

  // 清理空白字符串字段
  if (typeof next.model === "string" && !next.model.trim()) delete next.model;
  if (typeof next.thinking === "string" && !next.thinking.trim()) delete next.thinking;

  return next;
}
```

投递配置规范化

```
function coerceDelivery(delivery: UnknownRecord) {
  const next = { ...delivery };

  // "deliver" 模式重命名为 "announce"
  if (mode === "deliver") next.mode = "announce";

  // channel 和 to 字段的 trim 和 lowercase
  if (typeof delivery.channel === "string") {
    next.channel = delivery.channel.trim().toLowerCase();
  }

  return next;
}
```

默认值策略

`normalizeCronJobCreate` 和 `normalizeCronJobPatch` 的区别在于是否应用默认值：

```
// Create：应用默认值
export function normalizeCronJobCreate(raw) {
  return normalizeCronJobInput(raw, { applyDefaults: true });
}

// Patch：不应用默认值（只修改显式指定的字段）
export function normalizeCronJobPatch(raw) {
  return normalizeCronJobInput(raw, { applyDefaults: false });
}
```

关键默认值：

字段	默认值	条件
`wakeMode`	`"now"`	总是
`enabled`	`true`	总是
`sessionTarget`	`"main"`（systemEvent）/ `"isolated"`（agentTurn）	根据负载类型推断
`deleteAfterRun`	`true`	仅 `at` 类型（一次性任务执行后删除）
`delivery.mode`	`"announce"`	仅隔离 agentTurn 任务

本节小结

1. `CronService` 是 Cron 系统的门面类，所有逻辑委托给 `ops` 模块的纯函数
2. 通过**依赖注入**实现可测试性——时间源、日志、Agent 执行器均可注入
3. 所有操作通过 **Promise 链序列化**，避免异步并发导致的数据竞争
4. 支持三种调度类型：**一次性**（`at`）、**固定间隔**（`every`）、**Cron 表达式**（`cron`）
5. 调度引擎使用 **Croner 库**解析 Cron 表达式，定时器限制最大 **1 分钟延迟**以应对系统休眠
6. 时间解析支持 **Unix 毫秒**、**ISO 日期**、**ISO 日期时间**等多种格式，缺失时区默认 UTC
7. 规范化模块处理 LLM 生成的不可靠输入——大小写修正、旧版字段迁移、自动推断类型、默认值填充
8. `create` 和 `patch` 操作使用不同的规范化策略——create 应用默认值，patch 只修改显式字段